4 Classification

In Chapter 1, we examined various tasks that machine learning excels at, such as regression (predicting values) and classification (identifying classes). Having explored linear and polynomial regression in Chapter 2, we now turn our attention to classification in this section.

4.1 Binary Classifier

A binary classifier is a model that assigns inputs to one of two distinct classes. Linear binary classification is a simple but powerful method used to separate data into two distinct classes by drawing a straight decision boundary—such as a line (in 2D) or hyperplane (in higher dimensions)—between them. It works by computing a weighted sum of the input features and applying a threshold to determine the output class. Mathematically, it learns a weight vector θ and bias term b such that the decision rule can be written as:

$$\hat{y} = \begin{cases} 1 & \text{if } \theta^\top \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases} \tag{4.4}$$

where $\mathbf{x}$ is the input feature vector. During training, the weights are adjusted to minimize classification error, often using gradient descent on a suitable loss functions (e.g., hinge loss or logistic loss).

Despite its simplicity, the linear classifier is often surprisingly effective, especially when the data is linearly separable or nearly so. An example of such a case is shown in figure 4.1 where the three models (H_1, H_2, and H_3) are linear models defined by the hyperparameters θ that each correctly classify the data. The challenge then becomes how does one obtain the hyperparameters θ. To start, we will show that the hyperparameters θ can be obtained through gradient descent.

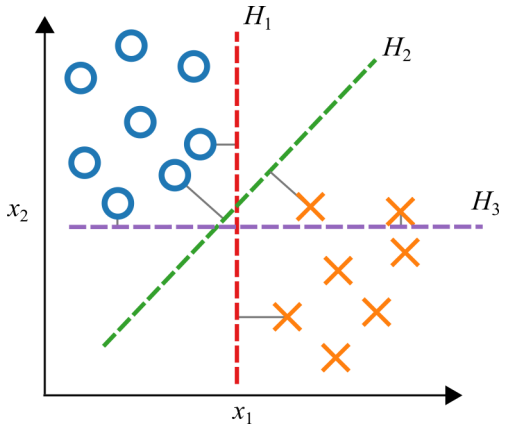


Figure 4.1: Linear classifier that can be implemented with gradient descent.

Review 4.1 MNIST (Modified National Institute of Standards and Technology) Dataset The MNIST (Modified National Institute of Standards and Technology) dataset is a collection of 70,000 small images of handwritten digits (figure 4.2). This dataset is known as "modified" because it combines two earlier sets of images: one created by U.S. high school students and another by US Census Bureau employees. First published in 1998, the dataset is particularly suitable for machine learning tasks as each image is labeled with the digit it represents. Each digit has been normalized and centered in a 28x28 pixel frame and anti-aliased to introduce grayscale levels. Each instance (i.e. picture) has 784 features that range from 0 to 256 and each represent an 8-bit gray scale pixel as shown in figure 4.3.

Typically, the dataset is split into two parts: the first 60,000 images are used for training, and the remaining 10,000 serve as validation data. It is standard practice to train and test models on the first 60,000 images, reserving the last 10,000 for final validation at the end of the project when the classifier is ready for deployment. Scikit-Learn includes a helper function that facilitates the downloading of the MNIST dataset.



Figure 4.2: A collection of the 10 digits (0-9) that make up the MNIST data set.

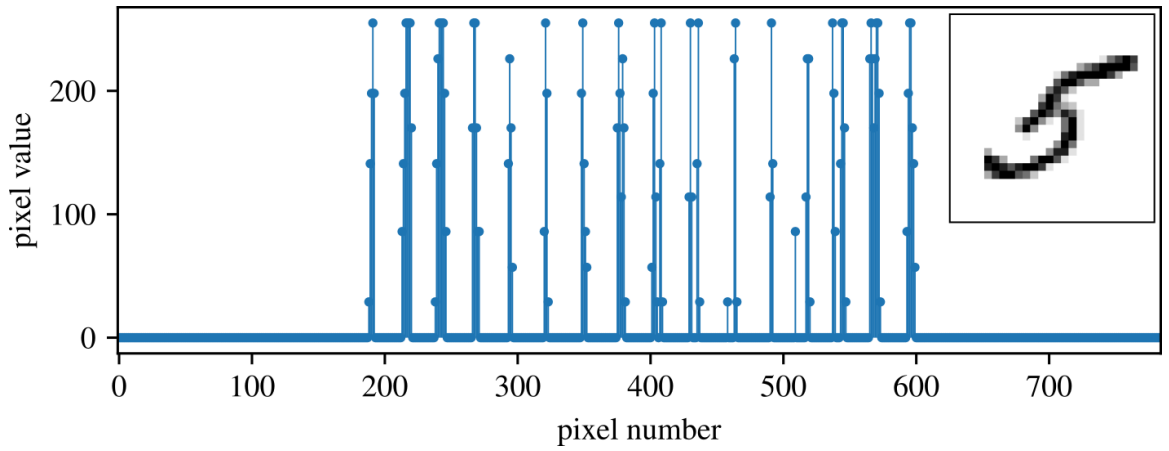


Figure 4.3: The 784 features that range from 0 to 256 and each represent an 8-bit gray scale pixel for a "5 digit" from the MNIST data set.

To begin, we simplify our task by focusing on identifying all the "5"s in the MNIST dataset. This task involves using a binary classifier that checks whether each digit is a 5. The number 5 is notoriously difficult to classify correctly within the MNIST dataset, making it a sensible starting point for binary classification.

Example 4.1 MNIST Dataset
This example demonstrates how to use `sklearn.datasets.fetch_openml` to load the MNIST dataset. It also includes visualization steps to display sample handwritten digits.

4.1.1 Regularized Linear Classifier

A practical algorithm for building this "5-detector" is the Regularized Linear Classifier solved with Stochastic Gradient Descent (SGD). The Regularized Linear Classifier offers an efficient and straightforward approach for discriminative learning of linear classifiers under convex loss functions, scaling well with large datasets and maintaining simplicity in implementation as it processes one training example at a time. The main advantages of the Regularized Linear Classifier include:

- Efficiency.
- Ease of implementation, providing many opportunities for optimization.

However, the Stochastic Gradient Descent classifier also presents certain disadvantages:

- It requires careful tuning of several hyperparameters, including the regularization parameter and the number of iterations.
- It is sensitive to feature scaling.

NOTE

The Regularized Linear Classifier solved with Stochastic Gradient Descent is commonly referred to simply called a Stochastic Gradient Descent classifier, despite Stochastic Gradient Descent only being the method used to trail the linear model. This is exemplified by the fact that scikit-learn uses the name `SGDClassifier` for their model that implements a "Regularized linear models with stochastic gradient descent (SGD) learning".

Given a labeled training set $(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^{m}$ with $y^{(i)} \in 1,0$ (1 marks the digit "5"), the regularised linear "5-detector" learns a weight vector θ (bias in the first entry θ_1) by minimising

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \max\left(0, 1 - y^{(i)}\theta^{\mathrm{T}}\mathbf{x}^{(i)}\right) + \frac{\lambda}{2}\left\|\theta_{1:}\right\|_2^2, \tag{4.5}$$

where the first term is the hinge loss enforcing a unit margin around the decision boundary. The second term applies ℓ_2-regularisation of strength $\lambda > 0$ to all weights except the bias (θ_1). Minimising (4.5) with stochastic-gradient descent yields a sparse, margin-maximising classifier that generalises well to unseen handwritten digits.

Example 4.2 Regularized Linear Classifier
This example trains a regularized linear classifier using `SGDClassifier` to distinguish the digit "5" from other digits in the MNIST dataset. It highlights how regularization helps improve generalization performance by penalizing large weights, and demonstrates basic prediction on a selected digit using a learned linear decision boundary.

4.2 Performance Measures for Binary Classification

Assessing the performance of a classifier can be more intricate than evaluating a regressor. Nonetheless, there are numerous performance metrics available that facilitate the comprehensive evaluation of a classifier's effectiveness.

4.2.1 Confusion Matrix

First, a brief explanation of Type I and Type II errors. These terms are not exclusive to classification problems in machine learning; they are also crucial in statistical hypothesis testing:

Type I Error: False positive (incorrectly rejecting a true null hypothesis)

Type II Error: False negative (incorrectly failing to reject a false null hypothesis)

These four cases from statistical testing can be combined into a single matrix known as the confusion matrix. In the context of our 5-detector, consider the simplified confusion matrix shown in figure 4.4.

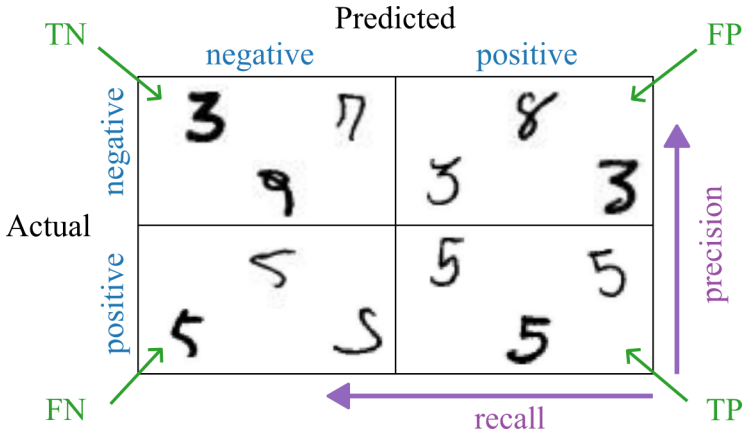


Figure 4.4: Confusion matrix binary.

A effective approach to evaluating classifier performance is to examine the confusion matrix. The essence is to tally the instances where class A is incorrectly classified as class B. A well-performing classifier will exhibit high values along the diagonal and lower values elsewhere in the matrix. Each row represents the actual class, while each column represents the predicted class. For instance, to determine how often the classifier misidentified images of 5s as 3s, inspect the intersection of the 5th row and 3rd column in figure 4.5.

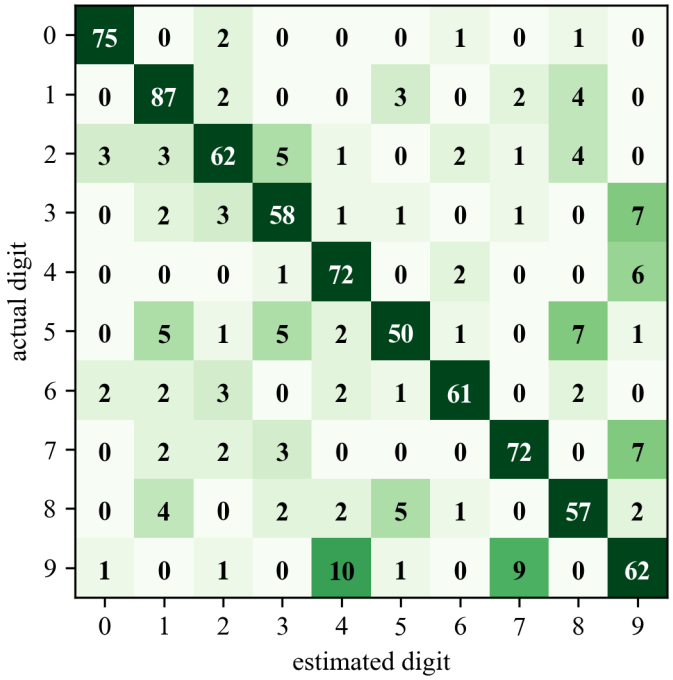


Figure 4.5: Confusion matrix for the first 1,000 data points from the MNIST dataset.

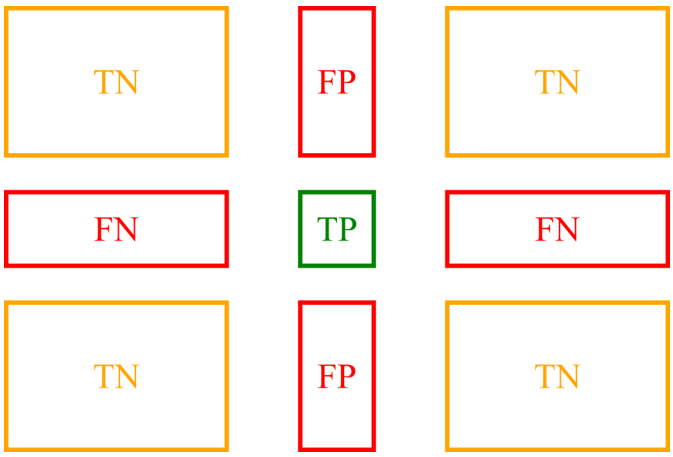


Figure 4.6: Confusion matrix breakout.

Each row in the confusion matrix corresponds to an actual class, while each column corresponds to a predicted class This is diagrammed in figure 4.6. The first row represents non-5 images (the negative class), and the second row represents images classified as 5s (the positive class). The first column of the first row indicates true negatives (TN), while the second column of the first row shows false positives (FP). The second row denotes images of 5s (the positive class): the first column represents false negatives (FN), and the second column displays true positives (TP). A perfect classifier would only have nonzero values along the main diagonal (from top left to bottom right).

Example 4.3 Binary Confusion Matrix
This example evaluates a binary classifier trained to detect the digit "5" using a confusion matrix. The matrix summarizes the number of true positives, true negatives, false positives, and false negatives. It also visualizes specific misclassifications to help interpret where the model fails, such as incorrectly labeling a "5" as not a "5" (false negative) or misidentifying another digit as a "5" (false positive).

4.2.2 Accuracy, Precision, and Recall

The confusion matrix provides detailed information, but sometimes a more concise metric is preferred. Before looking into accuracy, let's define it as

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total number of Predictions}} \tag{4.6}$$

or

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}. \tag{4.7}$$

While accuracy offers valuable insights, it's essential to consider the number of falsely classified positive and negative predictions, especially within the context of the problem being addressed. For instance, a 99% accuracy rate might seem satisfactory for predicting credit card fraud. However, what if a false negative indicates a severe virus or cancer? This is why we need to look deeper into the accuracy formula.

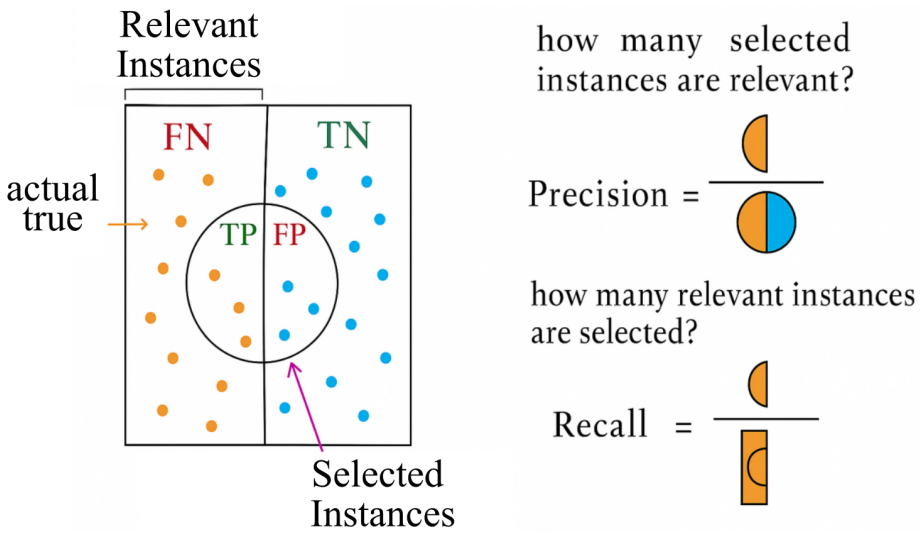


Figure 4.7: Visual representation of relevant instances, showing their relation to True Positives (TP) and False Positives (FP).

This is where precision and recall come into play. For this, we will need a definitions of instances

$$\text{Selected instances} = TP + FP \tag{4.8}$$

and

$$\text{Relevant instances} = TP + FN \tag{4.9}$$

which are visualized in figure 4.7. With these definitions in mind in mind, let's define precision as

$$\text{Precision} = \frac{\text{selected relevant instances}}{\text{selected instances}}, \tag{4.10}$$

or,

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{4.11}$$

One way to achieve perfect precision is to make only one positive prediction and ensure it's correct (precision = 1/1 = 100%). However, this approach would be impractical as it would ignore most positive instances. Recall complements precision by considering all relevant results, both true positives and false negatives. Recall, also known as sensitivity or true positive rate (TPR), is the ratio of positive instances correctly detected by the classifier. We can define recall as

$$\text{Recall} = \frac{\text{selected relevant instances}}{\text{relevant instances}}, \tag{4.12}$$

or,

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{4.13}$$

While the definition of accuracy, precision, and recall may not be immediately intuitive, a visualizations such as that shown in figure 4.7 can help clarify them.

To balance precision and recall, we often use the F1 score, which is the harmonic mean of precision and recall. The F1 score favors classifiers with similar precision and recall, computed as

$$F_1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN+FP}{2}}. \tag{4.14}$$

However, optimizing both precision and recall simultaneously is challenging due to the precision/recall tradeoff.

Figure 4.8 illustrates the tradeoffs when adjusting the decision threshold in a Regularized Linear Classifier. By adjusting the classification threshold, we can control the balance between precision and recall. Lowering the threshold increases recall but reduces precision, and vice versa.

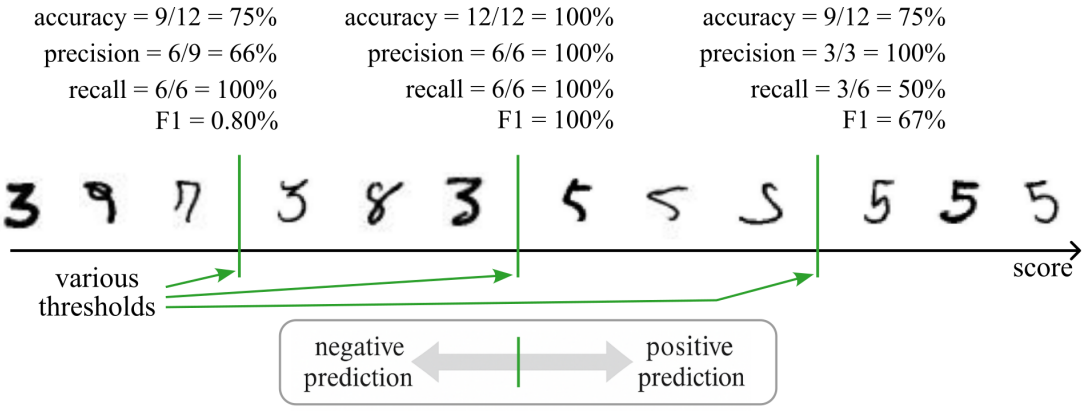


Figure 4.8: Visualization changing thresholds on the classification of digits in a "5-detector" and the resulting change in precision and recall.

To determine the optimal threshold, it's helpful to plot precision and recall against the threshold values. By default, the classifier in scikit-learn uses a threshold of zero, but this can be adjusted as needed. As illustrated in Figure 4.9, it's relatively straightforward to develop a classifier with nearly any desired precision: simply adjust the threshold to a sufficiently high value. However, it's important to bear in mind that a high-precision classifier may not be very practical if its recall is too low.

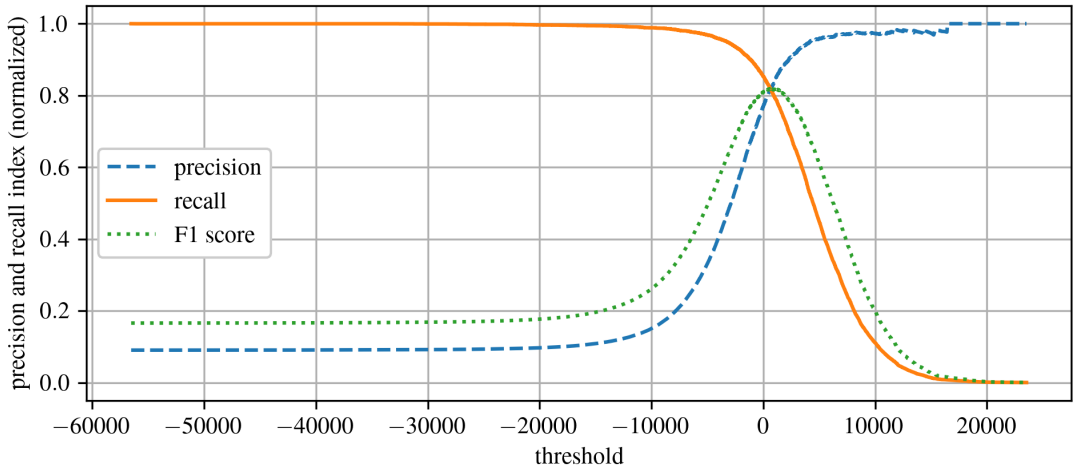


Figure 4.9: Visualization of the precision, recall, and F1 score as a function of changing a threshold value.

Figure 4.10 reports a precision vs recall curve that is obtained by changing the threshold of the trained model. At the leftmost part of the precision-recall curve (low recall), precision is highest because the model makes very few positive predictions, and those are likely true positives. As recall increases, the model starts predicting more positives, including more false positives, which reduces precision. The shape of the curve is also important; a steep drop-off indicates that the addition of false positives happens rapidly with increasing recall, whereas a more gradual decline suggests a better balance between precision and recall.

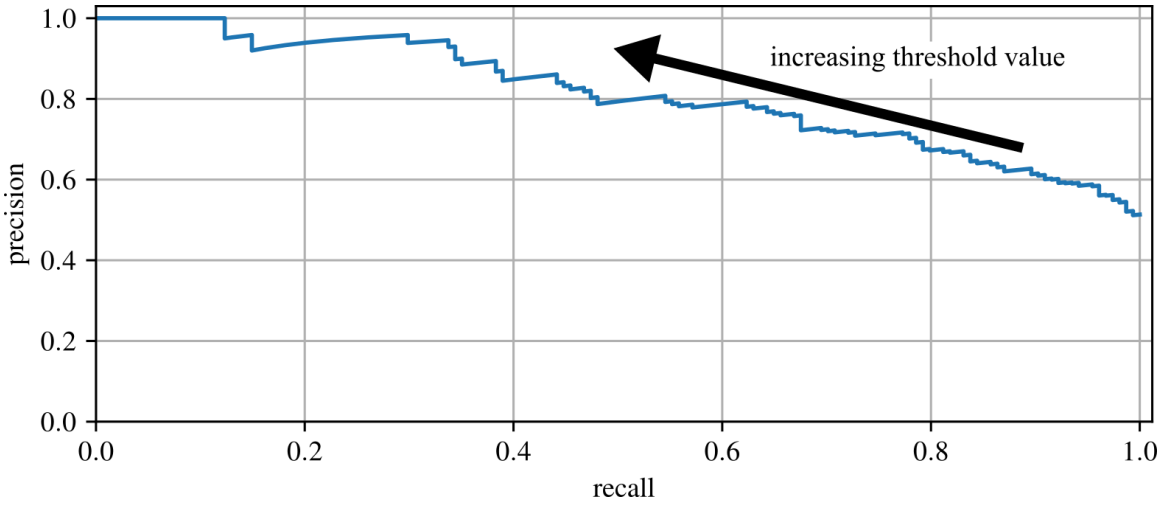


Figure 4.10: The relationship between precision vs recall for a changing threshold value.

By analyzing the curve, one can choose a threshold that balances precision and recall according to the specific needs of the application. For instance, in medical diagnosis, high recall is crucial to ensure no cases are missed, even if precision is lower. Additionally, the area under the precision-recall curve (AUC-PR) can be used to compare models. A higher area indicates a model with better performance across all thresholds.

Example 4.4 Accuracy, Precision, and Recall
This example computes and compares three key classification metrics (accuracy, precision, and recall) for a linear classifier trained to detect the digit "5" in the MNIST dataset. It shows both manual and `sklearn`-based calculations, introduces the F1 score, and visualizes how precision and recall vary with the classification threshold using a precision-recall curve.

4.3 k-fold Cross-validation

As you've likely observed, training your SGD classifier doesn't always yield consistent results in terms of model performance; hence the "stochastic" in stochastic gradient descent. One approach to obtaining a solution closer to the global minimum is to allow the gradient descent algorithm to iterate over more epochs. However, this poses a challenge as it's highly time-consuming and may lead to overfitting the model. Moreover, the variability in model performance makes it difficult to gauge the model's effectiveness.

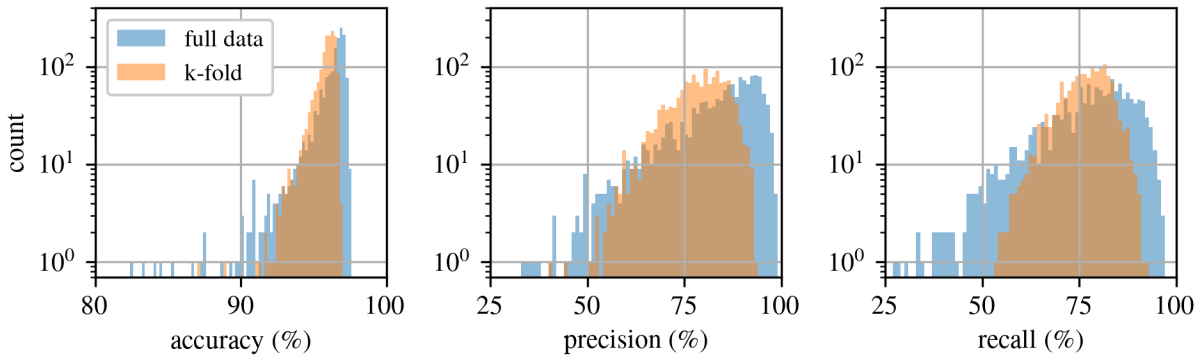


Figure 4.11: Performance metrics for 500 5-detectors trained either with full data or through k-fold cross-validation with 3 folds.

k-fold cross-validation offers a more rigorous method for evaluating your algorithm's performance. For instance, consider Figure 4.11, which depicts the performance metrics for 500 classification models trained as 5-detectors. Noticeably, the results obtained using k-fold cross-validation exhibit significantly less variability compared to training the classifier on the full dataset. In k-fold cross-validation, the training set is divided into smaller subsets for training and validation. The model is trained on these subsets and evaluated on the validation sets. Figure 4.12 provides a graphical representation of this technique.

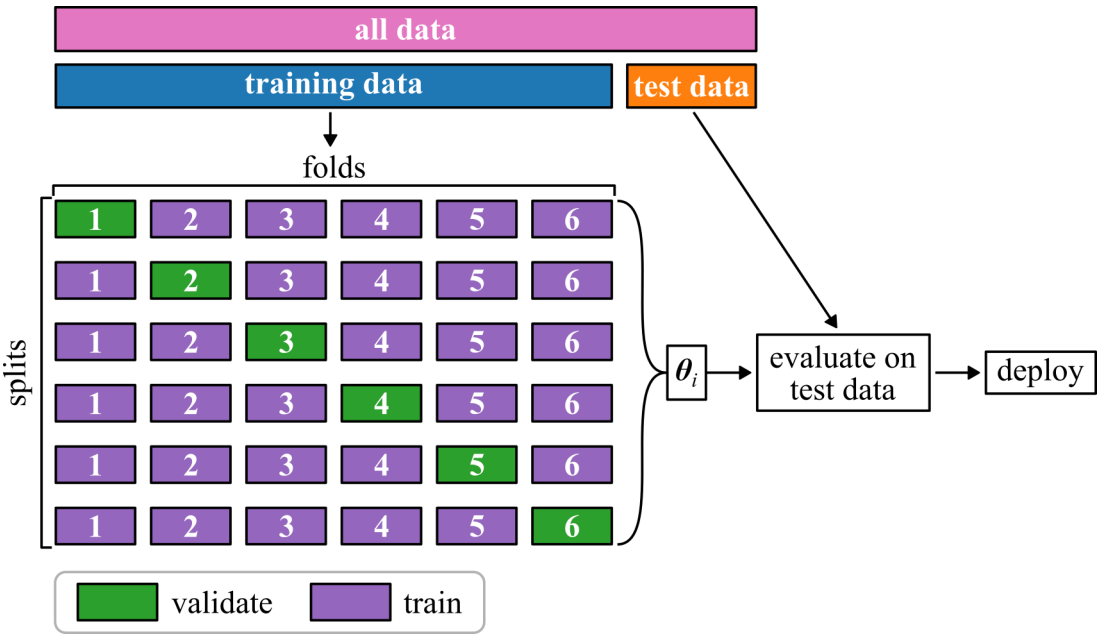


Figure 4.12: Illustration of how k-fold cross-validation operates.

Of course, Scikit-Learn provides a cross-validation feature, known as `sk.model_selection.cross_val_score`. This function conducts k-fold cross-validation by randomly partitioning the training set into distinct folds. Each fold is used for validation while the rest are used for training the SGD classifier. The reported performance metrics are averaged across all folds. It's important to note that by partitioning the data into folds, the number of samples available for learning is limited, which may result in peculiar classifiers with unusually high or low error rates. Despite being more computationally expensive, this approach effectively utilizes all the data for training and validation.

Example 4.5 ***k*-fold Cross-Validation**
This example trains a binary classifier to detect the digit "5" from the MNIST dataset using Stochastic Gradient Descent (SGD). It first demonstrates how model performance can vary when trained multiple times on the same dataset. Then, by applying k-fold cross-validation using `sk.model_selection.cross_val_predict`, it shows how splitting the data into k subsets helps reduce this variation and provides a more reliable estimate of accuracy, precision, and recall.

4.4 Multiclass Classification

Previously, we explored binary classifiers, which distinguish between two classes. Now, we'll explore the use of multiclass classifiers (also known as multinomial classifiers), which can discern between more than two classes. The simplest approach to multiclass classification involves multiple iterations of a binary classifier. There are two primary strategies for achieving this, both of which are considered in the context of the MNIST database challenge:

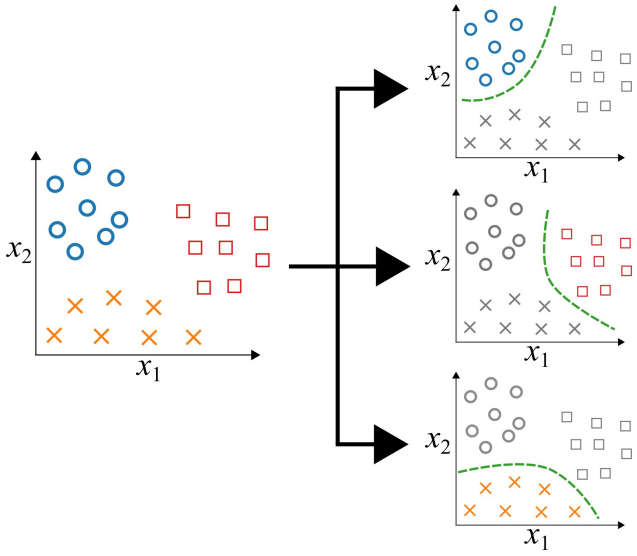


Figure 4.13: One-versus-Rest (OvR) classifier for a multi-class dataset.

- **One-versus-Rest (OvR)**: This strategy involves creating a system capable of classifying digit images into 10 classes (from 0 to 9) by training 10 binary classifiers, one for each digit (e.g., a 0-detector, a 1-detector, etc.). When classifying an image, the decision score from each classifier is obtained, and the class with the highest score is selected. While sometimes referred to as One-Versus-All (OvA), this term is slightly misleading as it does not involve comparisons against the same class.

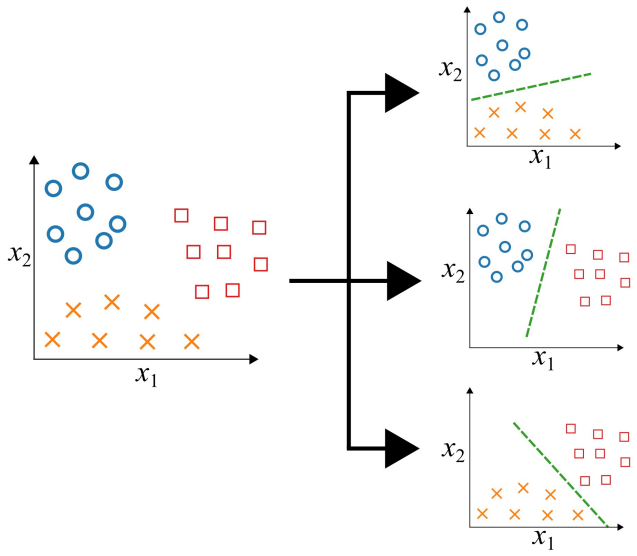


Figure 4.14: One-versus-one (OvO) classifier for a multi-class dataset.

- **One-versus-one (OvO)**: This approach trains a binary classifier for every pair of digits, such as one for distinguishing between 0s and 1s, another for distinguishing between 0s and 5s, and so forth. If there are N classes, $N \times (N-1)/2$ classifiers need to be trained. For

the MNIST problem, this translates to training 45 binary classifiers. During classification, the image is evaluated against all 45 classifiers, and the class with the most victories is selected. The primary advantage of OvO is that each classifier only needs to be trained on the relevant portion of the training set for the two classes it distinguishes, which is beneficial for algorithms that scale poorly.

Certain algorithms, such as Support Vector Machine classifiers, suffer from scalability issues with larger training sets. Algorithms that do not scale well with large datasets often resort to the OvO approach because training many small classifiers is computationally cheaper than fitting a handful of models on the full data. For most binary classifiers the OvR strategy is the norm. Some methods (including Random Forests and naïve Bayes) support multiclass problems natively and therefore require neither reduction strategy.

Example 4.6 Multiclass Regularized Linear Classifier
This example extends the use of `SGDClassifier` to multiclass classification on the MNIST dataset. It compares three approaches: One-vs-Rest (OvR), One-versus-One (OvO), and Scikit-Learn's default automatic strategy. Each method trains multiple binary classifiers to distinguish between the 10 digit classes.

4.5 Performance Measures for Multiclass Classification

Assessing a multiclass classifier often poses more challenges compared to evaluating a binary classifier. Nonetheless, many of the principles and methodologies utilized in evaluating binary classifiers can be seamlessly applied to multiclass classifiers as well.

4.5.1 Confusion Matrix

First, examine the confusion matrix. To do this, generate predictions using the `cross_val_predict()` function, followed by calling the `confusion_matrix()` function, similar to your previous steps. The confusion matrix in figure 4.15 appears satisfactory, with most images located on the main diagonal, indicating correct classification. However, the 5s appear slightly darker than other digits, suggesting fewer instances of 5s in the dataset or inferior performance of the classifier on 5s. Further investigation confirms both scenarios.

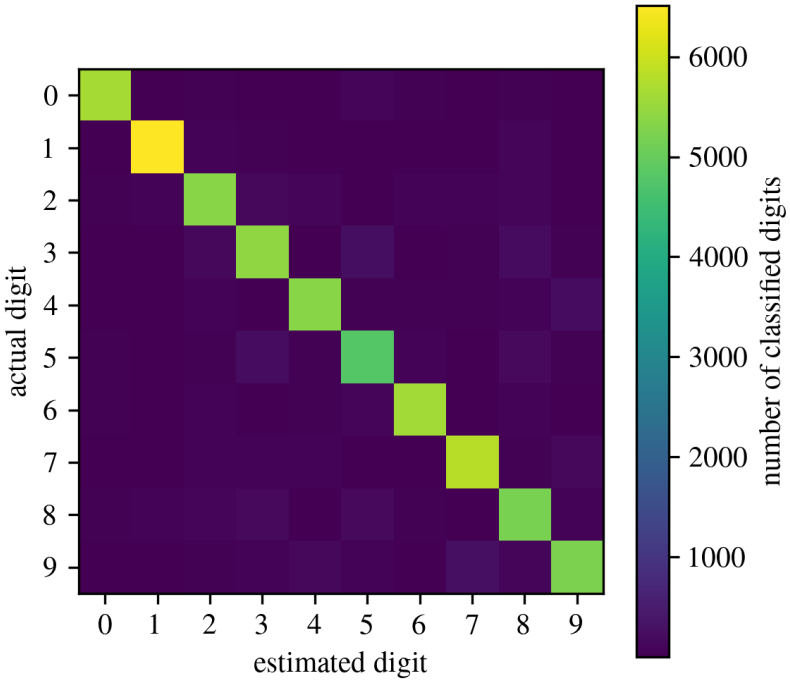


Figure 4.15: Confusion matrix for the MNIST data set solved using a one-versus-one classifier with Stochastic Gradient Descent and a k-fold of 3.

To turn figure 4.15 into a figure that highlights the mistakes, first normalize each entry of the confusion matrix by the number of images in its true class so you compare error rates rather than raw counts that would overweight frequent classes. Then set the diagonal cells to `NaN` so only the off-diagonal errors remain visible, and plot the resulting matrix. The resulting figure 4.16 more clearly highlights the errors in the confusion matrix.

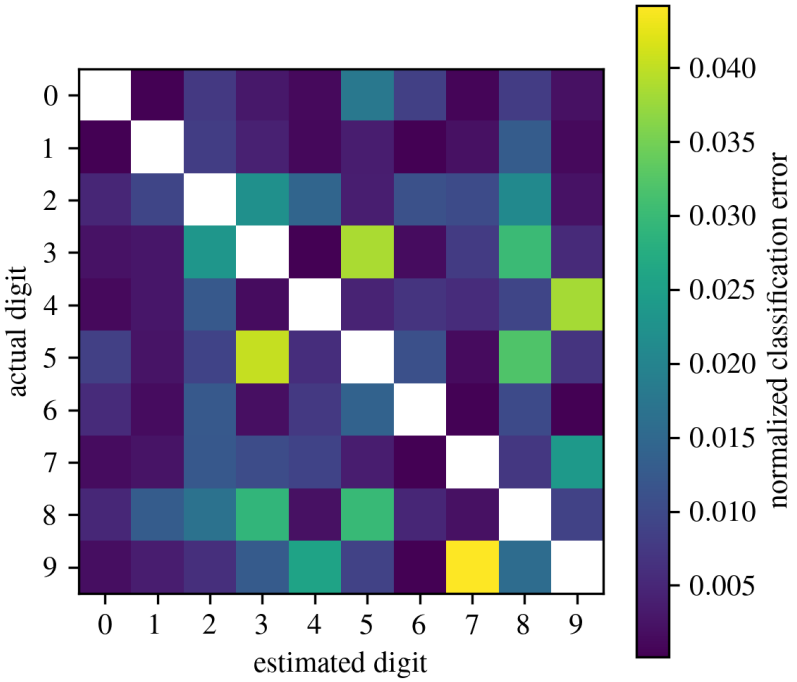


Figure 4.16: Normalized confusion matrix (similar to figure 4.15) with the diagonal removed to emphasize errors.

The refined plot facilitates a clear understanding of the classifier's error patterns. Rows represent actual classes, while columns depict predicted classes. Columns corresponding to classes 8 and 9 exhibit brightness, indicating numerous misclassifications as 8s or 9s. Likewise, rows for classes 8 and 9 also appear bright, signifying frequent confusion between 8s, 9s, and other digits. Conversely, some rows, like row 1, display darkness, indicating accurate classification of most 1s (with few exceptions confused with 8s). Notably, errors are asymmetric; for instance, more 5s are misclassified as 8s than vice versa.

Analyzing the confusion matrix provides valuable insights for classifier enhancement. In this case, efforts should concentrate on improving the classification of 8s and 9s, along with addressing the specific 3/5 confusion. Potential strategies include gathering more training data for these digits, devising new features (e.g., counting closed loops), or preprocessing images to emphasize distinguishing patterns (e.g., using Scikit-Image, Pillow, or OpenCV).

4.5.2 Analyzing Individual Errors

Analyzing individual errors provides valuable insights into the classifier's behavior and reasons for its failures.

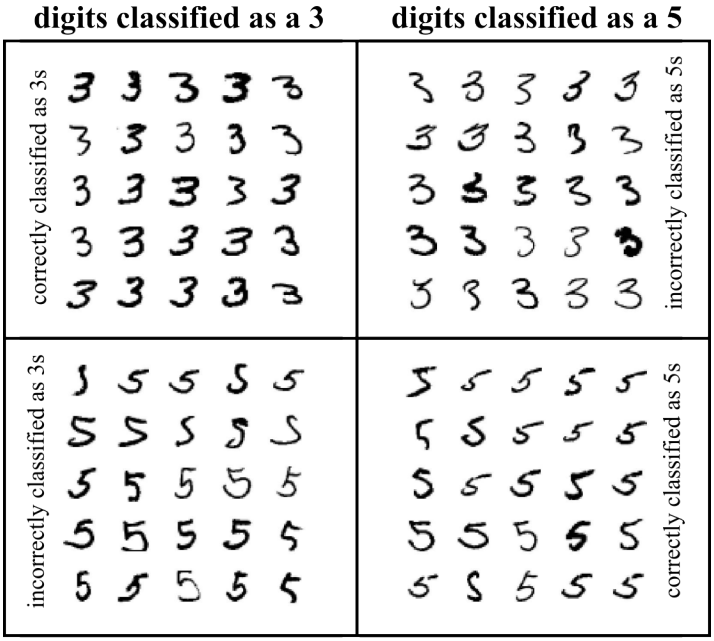


Figure 4.17: Confusion matrix showing digits classified as 3s and 5s for the same classifier used to obtain the results shown in figure 4.15.

The left half of figure 4.17 contains the blocks for images the classifier labelled as 3, while the right half shows those it labelled as 5. A few misclassified digits are so poorly written that even a human might hesitate, yet many errors look quite clear. Remember that a regularised linear classifier assigns a single weight to every pixel for each class and then sums the weighted pixel intensities to score each class.

Because just a handful of pixels separate a 3 from a 5, a linear classifier often confuses the two. Their main distinction is the small stroke that joins the top bar to the bottom curve; even a slight shift or rotation of this junction can flip the prediction. The linear model is therefore very sensitive to translations and rotations. Pre-processing the images to center the digits and correct their orientation should lessen the 3-5 confusion and improve performance across the board. 5

A non-linear model can learn more complex patterns by applying transformations (such as kernel mappings), effectively giving different relative importance to various parts of the image, and would therefore be better equipped to capture more intricate pixel patterns and more effectively separate these digits.

Example 4.7 Multiclass Confusion Matrix

This example uses a One-versus-One classifier trained with Stochastic Gradient Descent to evaluate multiclass classification performance on the MNIST dataset. It constructs a confusion matrix using 3-fold cross-validation and visualizes both the raw classification counts and the normalized error rates, helping identify which digits are most commonly misclassified.

4.6 Examples

Example 4.1

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 4.1 Load the MNIST data set
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import scipy as sp
import matplotlib as mpl
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import datasets

plt.close('all')


#%% Load your data

# this fetches "a" MNIST dataset  from openml and loads it into your environment
# as a Bunch, a Dictionary-like object that exposes its keys as attributes.
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')

# calling the DESCR key will return a description of the dataset
print(mnist['DESCR'])

# calling the data key will return an array with one row per instance and one
# column per feature where each features is a pixel, as defined in the key feature_names
X = mnist['data']


# calling the target key will return an array with the labels
Y = np.asarray(mnist['target'],dtype=int)

# Each image is 784 features or 28×28 pixels, however, the features must be reshaped
# into a 29x29 grid to make them into a digit, where the values represents one
# the intensity of one pixel, from 0 (white) to 255 (black).

digit_id = 35 # An OK 5
# digit_id = 0 # An odd 5
# digit_id = 100 # A bad 5


test_digit = X[digit_id,:]
digit_reshaped = np.reshape(test_digit,(28,28))

# plot an image of the random pixel you picked above.
plt.figure()
plt.imshow(digit_reshaped,cmap = mpl.cm.binary,interpolation="nearest")
plt .title('A "'+str(Y[digit_id])+'" digit from the MNIST dataset')
plt.xlabel('pixel column number')
plt.ylabel('pixel row number')


```

Example 4.2

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 4.2 Stochastic Gradient Descent (SDG) for the MINST data set
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import scipy as sp
import matplotlib as mpl
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import datasets

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')

#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')
X = mnist['data']    # load the data
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Train a Stochastic Gradient Descent classifier

# Extract a subset for our "5-detector".
Y_train_5 = (Y_train == 5)
Y_test_5 = (Y_test == 5)

# build and train the classifier
sgd_clf = sk.linear_model.SGDClassifier()
sgd_clf.fit(X_train, Y_train_5)

# get a digit from the dataset to test the classifier on
digit_id = 35 # An OK 5
# digit_id = 0 # An odd 5
# digit_id = 100 # A bad 5
test_digit = X[digit_id,:]
digit_reshaped = np.reshape(test_digit,(28,28))

# plot an image of the random pixel you picked above.
plt.figure()
plt.imshow(digit_reshaped,cmap = mpl.cm.binary,interpolation="nearest")
plt .title('A "'+str(Y[digit_id])+'" digit from the MNIST dataset')
plt.xlabel('pixel column number')
plt.ylabel('pixel row number')
plt.savefig('MNIST_digit')


# we can now test this for the "5" that we plotted earlier.
print(sgd_clf.predict([test_digit])) # a True case
print(sgd_clf.predict([X[2,:]])) # a False case


```

Example 4.3

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 4.3 Confusion matirx for the MINST dataset
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import scipy as sp
import matplotlib as mpl
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import pipeline
from sklearn import datasets
from sklearn import metrics

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')


#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')
X = mnist['data']    # load the data
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Train a Stochastic Gradient Descent classifier

# Extract a subset for our "5-dector".
Y_train_5 = (Y_train == 5)
Y_test_5 = (Y_test == 5)

# build and train the classifier
sgd_clf = sk.linear_model.SGDClassifier()
sgd_clf.fit(X_train, Y_train_5)

# we can now test this for the "5" that we plotted earlier.
digit_id = 35
test_digit = X[digit_id,:]
print(sgd_clf.predict([test_digit]))

#%% Build the Confusion Matrices

# Return the predictions made on each test fold
X_train_pred = sgd_clf.predict(X_train)


# build the confusion Matrix
print(sk.metrics.confusion_matrix(Y_train_5, X_train_pred))

# Now let's find all the False positive and false negative
confusion_booleans = np.vstack((Y_train_5, X_train_pred)).T
FN_index = np.where((confusion_booleans == [True,False]).all(axis=1))[0]
FP_index = np.where((confusion_booleans == [False,True]).all(axis=1))[0]

```

```
# We built a 5-detector, so:
# True and True is true positive (TP)
# False and False is True Negative (TN)
# True and False is false negative (FN)
# False and True is false positive (FP)


# from this, we see #0 is a false negative (FN), i.e. its is actually a 5 but
# the classifier said it was not. We can plot this digit below
digit_id = 0
test_digit = X[digit_id,:]
digit_reshaped = np.reshape(test_digit,(28,28))
plt.figure()
plt.imshow(digit_reshaped,cmap = mpl.cm.binary,interpolation="nearest")
plt .title('A "'+str(Y[digit_id])+'" digit from the MNIST dataset')
plt.xlabel('pixel column number')
plt.ylabel('pixel row number')

# Lastly, we see that #68 is classified as an false positive (FP), again, we can plot
this.
digit_id = 161
test_digit = X[digit_id,:]
digit_reshaped = np.reshape(test_digit,(28,28))
plt.figure()
plt.imshow(digit_reshaped,cmap = mpl.cm.binary,interpolation="nearest")
plt .title('A "'+str(Y[digit_id])+'" digit from the MNIST dataset')
plt.xlabel('pixel column number')
plt.ylabel('pixel row number')


```

Example 4.4

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 3.4 Precision and recall accuracy for the MNIST dataset
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import datasets
from sklearn import metrics

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')


#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')
X = np.asarray(mnist['data'])     # load the data
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Train a Stochastic Gradient Descent classifier

# Extract a subset for our "5-detector".
Y_train_5 = (Y_train == 5)
Y_test_5 = (Y_test == 5)

# build and train the classifier
sgd_clf = sk.linear_model.SGDClassifier()
sgd_clf.fit(X_train, Y_train_5)

#%% Build the Confusion Matrices

# Return the predictions made with the trained model
X_train_pred = sgd_clf.predict(X_train)

# build the confusion Matrix
confusion_matrix = sk.metrics.confusion_matrix(Y_train_5, X_train_pred)

TN = confusion_matrix[0,0]
FP = confusion_matrix[0,1]
FN = confusion_matrix[1,0]
TP = confusion_matrix[1,1]

#%% Calculate Precision and Recall

# calculate the Precision and Recall values using the commands discussed in class
accuracy = (TP + TN)/(TP + TN + FP + FN)
precision = TP/(TP+FP)
recall = TP/(TP+FN)

# of course, SK learn has built-in functions for this.
```

```
68  accuracy_SK = sk.metrics.accuracy_score(Y_train_5, X_train_pred)
69  precision_SK = sk.metrics.precision_score(Y_train_5, X_train_pred)
70  recall_SK = sk.metrics.recall_score(Y_train_5, X_train_pred)
71
72  # compute the F1 score for the data set
73  F1 = sk.metrics.f1_score(Y_train_5, X_train_pred)
74
75  #%% plot the precision and recall over the threshold domain.
76
77  # first, compute the scores for the predictions.
78  Y_scores= sgd_clf.decision_function(X_train)
79
80  #Y_scores= Y3
81  # now, for the scores and the target training set, calculate the precisions, recalls,
    threshold values.
82  precisions, recalls, thresholds = sk.metrics.precision_recall_curve(Y_train_5, Y_scores)
83
84  # now, compute the F1 score over the entire threshold range
85  F1s = 2/(1/precisions + 1/recalls)
86
87  # plot the Precision and Recall vs threshold.
88  plt.figure()
89  plt.plot(thresholds, precisions[:-1], "--", label="Precision")
90  plt.plot(thresholds, recalls[:-1], "-", label="Recall")
91  plt.plot(thresholds, F1s[:-1], ":", label="F1 score")
92  plt.xlabel("Threshold")
93  plt.ylabel("normalized precision\nand recall index")
94  plt.legend(loc=6,framealpha=1)
95  plt.ylim([-0.05, 1.05])
96  plt.grid(True)
97  plt.tight_layout()
98
99
100
101
102
103
104
105
106
107
108
109
```

Example 4.5

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 4.5 k-fold cross-validationStochastic for Gradient Descent (SDG) using the MINST
data set
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().magic('reset -sf')

import numpy as np
import scipy as sp
import pandas as pd
from scipy import fftpack, signal # have to add
import matplotlib as mpl
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import pipeline
from sklearn import datasets

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')


#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False)
X = mnist['data']    # load the data
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Train a Stochastic Gradient Descent classifier

# Extract a subset for our "5-detector".
Y_train_5 = (Y_train == 5)
Y_test_5 = (Y_test == 5)

sgd_clf = sk.linear_model.SGDClassifier()

#%% Build a 5-detector several times so see the variation in metrics returned by SGD

# Solve the model multiple times to see the variations
for i in range(5):

    # Train the model and eturn the predictions made by SGD
    sgd_clf.fit(X_train, Y_train_5)
    Y_train_pred = sgd_clf.predict(X_train)

    # Use SK learn model to retunr metrics
    accuracy = sk.metrics.accuracy_score(Y_train_5, Y_train_pred)
    precision = sk.metrics.precision_score(Y_train_5, Y_train_pred)
    recall = sk.metrics.recall_score(Y_train_5, Y_train_pred)
    print('accuracy is '+str(np.round(accuracy,4))+'; precision is '+str(np.round(
    precision,4))+
          '; recall is '+str(np.round(recall,4)))


#%% Build a 5-detector using k-fold cross-validation
```

```
66
67  # From our example we can see quite a variation in results for a model, making it
68  # hard to select the proper model. However, k-fold cross-validation can help with this.
69
70
71  # make a prediction using the k-fold method to split up the data set. Again,
72  # solve the model multiple times to see the variations
73  for i in range(5):
74
75      # Train the model and eturn the predictions made by SGD
76      Y_train_pred = sk.model_selection.cross_val_predict(sgd_clf, X_train, Y_train_5, cv=3
        )
77
78      # Use SK learn model to retunr metrics
79      accuracy = sk.metrics.accuracy_score(Y_train_5, Y_train_pred)
80      precision = sk.metrics.precision_score(Y_train_5, Y_train_pred)
81      recall = sk.metrics.recall_score(Y_train_5, Y_train_pred)
82      print('accuracy is '+str(np.round(accuracy,4))+'; precision is '+str(np.round(
        precision,4))+
83            '; recall is '+str(np.round(recall,4)))
84
85
```

Example 4.6

```
"""
Example 4.6 Multiclass Stochastic Gradient Descent (SDG) for the MINST data set
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
import time as time
from sklearn import linear_model
from sklearn import pipeline
from sklearn import datasets
from sklearn import multiclass

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')

#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')
X = np.asarray(mnist['data'])   # load the data
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Train a Multiclass Stochastic Gradient Descent classifiers

# SK learn has a  Multiclass and multilabel module as sk.multiclass. You can use
# this module to do one-vs-the-rest or one-vs-one classification.

# here we test a one-vs-rest classifier that uses Stochastic Gradient Descent
tt_1 = time.time()
ovr_clf = sk.multiclass.OneVsRestClassifier(sk.linear_model.SGDClassifier())
ovr_clf.fit(X_train, Y_train)
print('One-vs-Rest took '+str(time.time()-tt_1 )+' seconds to train and execute')

# here we test a one-vs-one classifier that uses Stochastic Gradient Descent
tt_1 = time.time()
ovo_clf = sk.multiclass.OneVsOneClassifier(sk.linear_model.SGDClassifier())
ovo_clf.fit(X_train, Y_train)
print('One-vs-one took '+str(time.time()-tt_1 )+' seconds to train and execute')

# Moreover, Scikit-Learn detects when you try to use a binary classification algorithm
for
# a multiclass classification task, and it automatically runs OvA (except for SVM
classifiers for which it uses OvO).
tt_1 = time.time()
multi_sgd_clf = sk.linear_model.SGDClassifier()
multi_sgd_clf.fit(X_train, Y_train) # y_train, not y_train_5
print('SK learns automated selection (OvA) took '+str(time.time()-tt_1 )+' seconds to
train and execute')
```

Example 4.7

```
"""
Example 4.7 Multiclass confusion matrix for the MINST data set
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk

cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
plt.close('all')

#%% Load your data

# Fetch the MNIST dataset from openml
mnist = sk.datasets.fetch_openml('mnist_784',as_frame=False,parser='auto')
X = np.asarray(mnist['data'])    # load the data and convert to np array
Y = np.asarray(mnist['target'],dtype=int)  # load the target

# Split the data set up into a training and testing data set
X_train = X[0:60000,:]
X_test = X[60000:,:]
Y_train = Y[0:60000]
Y_test = Y[60000:]

#%% Confusion Matrix for a Multiclass classifier.

# Use the one-vs-one classifier that uses Stochastic Gradient Descent as this is
# faster for this specific data set
ovo_clf = sk.multiclass.OneVsOneClassifier(sk.linear_model.SGDClassifier())

# make a prediction for every case using the k-fold method.
Y_train_pred = sk.model_selection.cross_val_predict(ovo_clf, X_train, Y_train, cv=3)
conf_mx = sk.metrics.confusion_matrix(Y_train, Y_train_pred)
print(conf_mx)

# plot the results
fig = plt.figure(figsize=(4,4))
pos = plt.imshow(conf_mx) #, cmap=plt.cm.gray)
cbar = plt.colorbar(pos)
cbar.set_label('number of classified digits')
plt.ylabel('actual digit')
plt.xlabel('estimated digit')
plt.savefig('confusion_matrix',dpi=300)

# Normalize the confusion matrix by class size to compare error rates, not raw counts.
row_sums = conf_mx.sum(axis=1, keepdims=True)
norm_conf_mx = conf_mx / row_sums

# Next, we remove the high values along the diagonal. This is done by converting the
# confusion matrix to a float data type, and replacing everything on the diagonal with
NaNs.
conf_mx_noise = np.asarray(norm_conf_mx,dtype=np.float32)
np.fill_diagonal(conf_mx_noise, np.NaN)

# plot the results but only consider the noise
fig = plt.figure(figsize=(4,4))
pos = plt.imshow(conf_mx_noise) #, cmap=plt.cm.gray)
cbar = plt.colorbar(pos)
cbar.set_label('normalized classification error')
plt.ylabel('actual digit')
plt.xlabel('estimated digit')
plt.savefig('confusion_matrix_error',dpi=300)
```